

```
if (execve("my new program", ...) < 0)
{
    printf("execve failed\n");
}
else
    printf("execve succeeded\n");
}
```

The line ‘execve succeeded’ will not be output, because if the `execve` call is successful, it will execute the new program, overwriting the text, data, `bss` segment, and stack of the calling process. Hence, the `printf` would not be executed.

**Q** If a process is multi-threaded, and one of the threads invokes the `fork` system call, what happens?

The child process is a replica of the calling thread of the parent process, including its entire address space, and including the state of the mutex and all its resources.

**Q** If a process is multi-threaded, and one of the threads invokes the `exec` system call, what happens?

If the calling process is multi-threaded, a call to the `exec` function will cause all threads and light-weight processes (LWPs) in the calling process to be terminated, and the new executable image to be loaded and executed. No thread-specific data destructor functions are called. If the `exec` function fails and returns to the caller, the threads and LWPs in the calling process will not be terminated.

**Q** What is copy-on-write, and how is it used to reduce the fork-exec overhead?

Copy-on-write is an optimisation used to reduce the overhead of regular `fork` system calls, in cases where a child is going to invoke an `exec` system call immediately after the `fork` system call. In this case, the data and stack regions of the parent’s address space are marked as copy-on-write. The child process shares the data and stack region pages of the parent. If, after the `fork` system call, there is any modifying access to the data and stack region pages, then a page protection fault is raised, since these pages are now marked as copy-on-write. The fault handler makes a new, writeable, copy of the page, corresponding to the faulting access, and updates the address mapping for that page in the process requesting the modification. Thus, with copy-on-write, the data and stack pages of the parent are copied on demand, in a lazy fashion, instead of having to copy the entire address space at the time of the `fork` itself. If the child process invokes the `exec` system call, it results in revoking the copy-on-write for the shared data and stack pages of the parent.

**Q** Assume that a process detects and handles a stack overflow by protecting its stack region boundary with a write-protected page. Hence, any write to this page due to a stack overflow will result in a SIGSEGV being generated. Since the process stack has already overflowed, how can the

signal handler for the SIGSEGV be executed?

Many operating systems provide an alternate signal stack to address this issue. There is a system call known as *signalstack*, which allows a process to define an alternate signal handling stack for specific signals. Signals that have been explicitly declared to execute on the alternate stack will be delivered on the alternate stack.

**Q** Should threads be supported at the user level, at the kernel level, or both? Justify your answer.

Threads can be supported at different levels, either at the application level by the user-level threads library; at the kernel level by means of kernel threads; or a combination of the two. Simply providing user-level threads without the underlying kernel threads prevents parallel execution of multiple user threads at the same time, resulting in loss of concurrency. On the other hand, providing only kernel threads, without user-level threads, makes it difficult to extract parallelism, since the kernel does not have exposure to internal application-specific knowledge on what operations can be parallelised. Hence, a mix of kernel- and user-level threads is required.

## Computer architecture questions

**Q** What are write-through and write-back caches? What are their advantages and disadvantages?

In the case of a write-through cache, the processor updates the main memory in sync with the cache entry update. Both the cache entry and the main memory location are updated simultaneously. Every write issued by the processor to the cache results in a corresponding write to the main memory.

In a write-back cache, the processor does not update the main memory immediately, at the time of writing to the corresponding cache entry. Instead, the main memory location is updated only when the cache entry is evicted from the cache. The processor marks the written cache entries using a dirty bit. When a cache entry is selected for replacement, if it’s dirty bit is set, the processor updates the main memory location corresponding to the cache entry.

If a memory location (cache entry) is updated multiple times by a processor before it is evicted, then a write-back cache reduces the memory traffic, since only one write to main memory happens at the time of eviction. On the other hand, when a cache miss occurs in a write-back cache, and a cache entry with dirty bit set is selected for eviction, it first needs to be written back to main memory, before the newly requested data can be brought into the cache. This results in two memory requests: one to update the memory location corresponding to the entry getting evicted, and another to bring back the requested data. This increases the cache miss service time for write-back caches.

**Q** What are the advantages and disadvantages of associative caches over direct-mapped caches?

In the case of a direct-mapped cache, there is only one